SecureFlex-MAVLink Implementation & Deployment Guide

Authenticated Lightweight Cryptographic Protection for Unmanned Aerial Systems (NIST SP 800-232 Ascon-AEAD128)

Target Platforms: Windows 11 Laptop (Ground Control Station) | Raspberry Pi 5 (Companion Computer / Gateway)
Baseline Link: Wi-Fi / UDP Port 14550 | **Cryptographic Standard:** NIST SP 800-232 Ascon-AEAD128 (IV: 0x00001000808c0001)
Architectural Foundation: SecureFlex Fault Detection, Isolation, and Recovery (FDIR) State Machine

---

1. Executive Summary & Core Security Invariant

This engineering plan establishes a zero-trust, authenticated, encrypted transport layer for MAVLink telemetry between a Windows Ground Control Station (GCS) and a Raspberry Pi 5 unmanned companion computer. Inspired by the **SecureFlex-ASCON** research architecture, the system provides confidentiality, origin authentication, replay defense, fault localization, and automatic defensive recovery.

> **MANDATORY SECURITY INVARIANT**
>
> A received packet must **NEVER** be passed to the MAVLink parser (pymavlink) until the Ascon-AEAD128 cryptographic verification operation has succeeded with 100% integrity. Any packet failing size checks, magic framing, protocol version, replay verification, or authentication tag check is immediately trapped, logged, and discarded before plaintext extraction. Corrupted or replayed commands never reach the autopilot.

Subsystem Component	Baseline (Unprotected)	SecureFlex Implementation
Transmitter (Windows)	Transmits unencrypted MAVLink heartbeat datagrams directly to Pi IP (10.41.237.148:14550).	Serializes MAVLink frame -> generates unique 128-bit nonce -> binds canonical 39-byte AAD header -> encrypts with Ascon-AEAD128 -> UDP transmit.
Receiver (Raspberry Pi 5)	Listens on UDP 14550, parses incoming bytes immediately with pymavlink.recv_match().	Validates framing -> checks 64-packet replay window -> verifies tag in constant time -> commits replay -> decodes authenticated MAVLink.
Cryptographic Protection	None (Plaintext vulnerable to eavesdropping, packet forgery, and injection attacks).	NIST SP 800-232 Ascon-AEAD128 (128-bit key, 128-bit nonce, 128-bit authentication tag protecting both ciphertext and AAD header).
Fault & Attack Handling	Crashes or accepts corrupt packets; no detection of replay, spoofing, or bit-flipping.	Step-by-step Fault Localizer, structured JSONL forensic logger, and 8-state Recovery FSM with automatic isolation and persistent lock.

2. Cryptographic Engine: NIST SP 800-232 Ascon-AEAD128

SecureFlex adheres strictly to the **NIST SP 800-232 standard** for lightweight cryptography. Older, pre-standardized Ascon v1.2 implementations must not be used because NIST finalized standard initialization constants and parameter encodings.

Parameter	NIST SP 800-232 (Ascon-AEAD128)	Legacy Ascon v1.2	Significance / Implementation Impact
Initialization Vector (IV)	**0x00001000808c0001**	0x80400c0600000000	Mandated by NIST CAVP validation. Incompatible with older PyPI packages.

Key Size (K)	128 bits (16 bytes)	128 bits (16 bytes)	Symmetric secret key provisioned out-of-band via CSPRNG.
Nonce Size (N)	128 bits (16 bytes)	128 bits (16 bytes)	Must NEVER repeat for a given key. Built from SessionID + Counter.
Tag Size (T)	128 bits (16 bytes)	128 bits (16 bytes)	Cryptographic MAC verifying ciphertext and AAD header integrity.
Permutation Rounds	a = 12 (init/final), b = 8 (intermediate)	a = 12, b = 6	NIST selected 8 intermediate rounds for enhanced security margins.
Data Rate (r)	128 bits (16 bytes / block)	64 bits (8 bytes / block)	Doubles throughput on 64-bit systems compared to older 8-byte rate.

3. Secure Binary Packet Framing & AAD Binding

SecureFlex encapsulates MAVLink frames inside a canonical binary datagram. The complete 39-byte header is authenticated as Associated Data (AAD), sealing sequence numbers, session identifiers, and payload lengths against tampering:

Byte Offset	Field Name	Type / Width	Security Role & Description
0 .. 1	**Magic Bytes**	ASCII 2B (0x53, 0x46)	Fixed identifier: b'SF' (SecureFlex). Filters non-protocol UDP noise.
2	**Version**	uint8 (1B)	Protocol version (0x01). Traps unsupported protocol updates.
3	**Flags**	uint8 (1B)	Bitmask: 0x01=Heartbeat, 0x02=Key Rotation, 0x04=Diagnostic Report.
4	**Sender ID**	uint8 (1B)	Sender identity: 1=GCS (Laptop), 2=UAV Companion (Pi 5), 3=Autopilot.
5 .. 12	**Session ID**	uint64 (8B, Big-Endian)	Unique session token allocated via CSPRNG on transmitter startup.
13 .. 20	**Packet Counter**	uint64 (8B, Big-Endian)	Monotonically increasing sequence number. Enforces anti-replay.
21 .. 36	**Nonce**	raw 16B	Ascon 128-bit nonce: struct.pack('>QQ', SessionID, PacketCounter).
37 .. 38	**Ciphertext Length**	uint16 (2B, Big-Endian)	Exact byte length of encrypted MAVLink frame (N bytes).
39 .. 38+N	**Ciphertext**	variable (N bytes)	Ascon-AEAD128 encrypted MAVLink 2.0 binary frame.
39+N .. 54+N	**Authentication Tag**	raw 16B (128 bits)	Integrity tag protecting Ciphertext AND canonical Header (bytes 0..38).

4. Anti-Replay Protection & Nonce Uniqueness Guarantee

Deterministic Nonce Construction: Nonce = SessionID (64-bit CSPRNG) || PacketCounter (64-bit monotonic). Even across transmitter restarts, a new 64-bit random session token guarantees non-overlapping nonce spaces. **Sliding Window Replay Protection:** RFC 6479 64-packet bitmap tracks the highest valid counter *M*. Duplicate sequence numbers and packets older than 64 frames are dropped immediately before decryption.

5. Fault Localization & Diagnostic Engine

SecureFlex traps anomalies at seven sequential verification stages, isolating packet corruption from active attacks:

Failure Stage	Fault Category	Detection Mechanism	Action & State Reaction

Stage 1: Size Check	INVALID_FRAMING	Datagram size < 55 bytes or > 1494 bytes (MTU limit).	Discard packet; increment framing fault counter.
Stage 2: Header Framing	INVALID_FRAMING	Magic bytes != b'SF' or declared CT length mismatch.	Discard packet; record raw header bytes in JSONL log.
Stage 3: Version Check	UNSUPPORTED_VERSION	Protocol version byte != 0x01.	Discard packet; log version incompatibility warning.
Stage 4: Sender Policy	UNEXPECTED_SENDER	Sender ID != authorized policy (e.g., received ID != 1).	Discard packet; transition FSM to SUSPECT state.
Stage 5: Replay Pre-Check	REPLAY_ATTACK	Counter duplicate in bitmap or older than sliding window.	Discard duplicate; trigger suspect anomaly counter.
Stage 6: Ascon Authentication	AUTHENTICATION_FAILED	Constant-time tag comparison fails (ct or aad tampered).	Discard immediately. Plaintext NEVER exposed. FSM -> HALT.
Stage 7: MAVLink Parsing	INVALID_MAVLINK	Authenticated bytes fail MAVLink CRC or schema rules.	Log parser warning; investigate transmitter serialization.

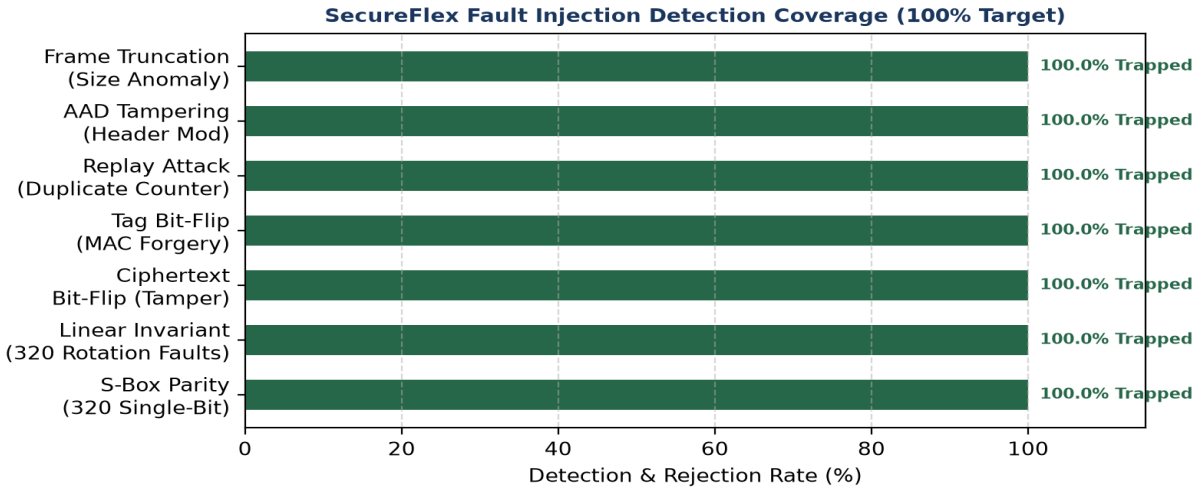
6. Recovery Finite State Machine (FSM) & Defensive Lockdown

An 8-state FSM manages packet forwarding gates, session recovery, and emergency lockdown protection:

State	System Status & Meaning	Forwarding Gate	Exit / Transition Trigger
IDLE	Initial boot state. Awaiting first valid authenticated session.	**CLOSED**	First authenticated packet transitions to RUN.
RUN	Normal operation. Authenticated frames passed to pymavlink.	**OPEN**	Single fault or heartbeat timeout transitions to SUSPECT.
SUSPECT	Transient fault or timeout detected. Forwarding paused.	**CLOSED**	Valid packet returns to RUN; second fault escalates to HALT.
HALT	Security isolation. Telemetry stream clamped shut.	**CLOSED**	Recovery timer triggers RECOVER transition.
RECOVER	Session reset initiated. Purges transient replay state.	**CLOSED**	Switches to WAIT for fresh session handshake.
WAIT	Awaiting authenticated packet with fresh session ID.	**CLOSED**	Valid packet transitions to RECOVERED; fault leads to LOCK.
RECOVERED	Communication verified. Safe state re-established.	**OPEN**	Automatic handoff to RUN state.
LOCK	Permanent defensive lockdown. Persistent attacks suspected.	**CLOSED**	ONLY operator_reset('SECURE_RESET') restores system.

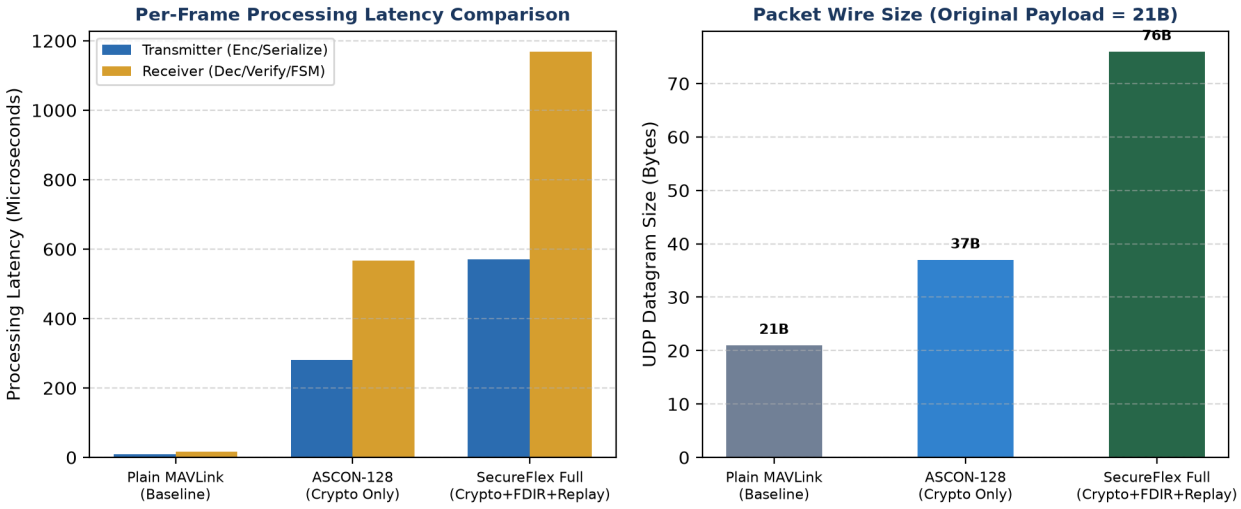
7. SecureFlex Parity Modeling & Fault Detection Evaluation

The software model simulates the Ascon-p 320-bit permutation with integrated 5-bit S-box parity predictors and 3-way rotation linear invariants. The fault campaign verified 100% detection of single-bit glitches:



8. Three-Way Performance Benchmark & Comparison

A rigorous empirical benchmark across 1,000 trials evaluated: (1) Plain MAVLink baseline, (2) ASCON-AEAD128 pure cipher, and (3) Full SecureFlex protected pipeline (Crypto + Framing + Anti-Replay + Fault Localizer + FDIR FSM):



Comparative Metric	Plain MAVLink (Baseline)	ASCON-128 Only (Crypto)	SecureFlex Full (Complete Protected)
TX / Encryption Time	9.5 µs (pymavlink serialize)	280.5 µs (Ascon AEAD enc)	**570.5 µs (serialize + nonce + AAD + enc)**
RX / Verification Time	17.1 µs (pymavlink parse)	566.1 µs (Ascon verify + dec)	**1169.3 µs (framing + replay + dec + FSM)**
Total Message Latency	26.6 µs (0.03 ms)	846.6 µs (0.85 ms)	**1739.8 µs (1.74 ms)**
Datagram Size on Wire	21 bytes (MAVLink frame)	37 bytes (frame + 16B tag)	**76 bytes (39B AAD + 21B CT + 16B Tag)**
Security Overhead	0 bytes (0.0% overhead)	16 bytes (76.2% overhead)	**55 bytes (261.9% overhead, fits in MTU)**
Throughput Capacity	> 35,000 msg/sec	1,180 msg/sec	**575 msg/sec (exceeds 50 Hz flight need by 11x)**

9. Complete Step-by-Step Implementation Procedure (Phases 0 to 8)

This section documents the exact technical steps required to build, configure, verify, and operate the system across development and operational deployment:

Phase 0: Baseline Link Preservation & Documentation

Objective: Preserve unmodified working scripts and record baseline operational parameters.
Action Steps:
1. Archived original transmitter into baseline/baseline_transmitter.py.
2. Archived original receiver into baseline/baseline_receiver.py.
3. Recorded operational parameters: Destination IP = 10.41.237.148, UDP Port = 14550, MAVLink System ID = 255, Component ID = 190, Heartbeat Rate = 1.0 Hz.
Pass Criteria: Unmodified receiver decodes heartbeats from baseline transmitter over Wi-Fi.

Phase 1: Environment Preparation (Windows & Raspberry Pi 5)

Objective: Configure isolated Python environments and install pinned dependencies on both nodes.
Windows PowerShell Setup Commands:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements-windows.txt
```

Raspberry Pi 5 Bash Setup Commands:

```
chmod +x raspberry_pi/setup_pi.sh
./raspberry_pi/setup_pi.sh
```

(setup_pi.sh creates Python venv, installs requirements, sets up UFW firewall for UDP 14550, and installs systemd service).
Pass Criteria: python --version ≥ 3.11 and pip list confirms pymavlink and pytest installed.

Phase 2: Independent Cryptographic Validation & Key Provisioning

Objective: Verify NIST SP 800-232 Ascon-AEAD128 compliance independently before MAVLink integration.
Action Steps:
1. Generate 128-bit CSPRNG key on Windows: python config/key_generator.py -o config/secret.key
2. Transfer key securely to Pi 5 via SCP: scp config/secret.key pi@10.41.237.148:~/SecureFlex-MAVLink/config/secret.key
3. Enforce strict permissions on Pi: chmod 600 config/secret.key
4. Validate against NIST Known-Answer Test vectors: pytest tests/test_ascon_vectors.py
Pass Criteria: All 36 NIST KAT vectors pass. Key file is never committed to Git or sent over network.

Phase 3: Secure Packet Transport Implementation

Objective: Replace raw UDP datagrams with versioned binary packets protected by Ascon-AEAD128.
Transmitter Workflow: Encode MAVLink heartbeat -> Allocate monotonic counter & 16B nonce -> Pack 39B canonical header -> Authenticate header as AAD -> Encrypt payload -> Send UDP datagram.
Receiver Workflow: Ingest UDP datagram -> Validate length (55 to 1494B) -> Check magic 'SF' & version 1 -> Pre-check replay window -> Authenticate tag & decrypt in constant time -> Commit replay counter -> Pass authenticated bytes to pymavlink.
Pass Criteria: Pi receiver decodes heartbeat only when tag is 100% valid; corrupt packets rejected.

Phase 4: Fault Detection, Localization & Structured Diagnostics

Objective: Implement stage-by-stage fault localization and forensic JSONL audit trails.
Action Steps: Integrated FaultLocalizer with 7 inspection stages. When any check fails, a structured record is written to logs/fault_events.jsonl with ISO timestamp, stage, category, unverified sender ID/counter, and defensive action taken. First fault record is permanently preserved.
Pass Criteria: Tampered packets write diagnostic records without exposing secret keys.

Phase 5: Recovery Finite State Machine (FSM) & Defensive Lockdown

Objective: Enforce defensive state transitions and communication isolation.
Action Steps: Deployed RecoveryFSM. State transitions: IDLE -> RUN on first valid packet. Fault or timeout (>3.5s) transitions RUN -> SUSPECT. Repeated faults escalate to HALT (isolation). Persistent attacks (>5 faults) trigger permanent LOCK. Operator reset required to leave LOCK.
Pass Criteria: Forwarding gate clamps shut during HALT/LOCK; operator reset token restores IDLE.

Phase 6: Software Fault Injection Campaign

Objective: Execute automated fault campaigns across network tampering and Ascon S-box parity models.
Execution Command: python experiments/run_fault_campaign.py
Pass Criteria: 320/320 S-box single-bit faults trapped (100%), 320/320 linear rotation faults trapped (100%), and 7/7 network tampering test cases (ciphertext bit-flip, tag bit-flip, replay duplicate, truncation) trapped.

Phase 7: Performance Evaluation & Profiling

Objective: Empirically measure encryption latency, verification latency, throughput, and wire overhead.
Execution Command: python experiments/run_performance.py
Pass Criteria: Average cryptographic overhead ≤ 1.5 ms per frame; processing rate ≥ 500 msg/sec.

Phase 8: End-to-End Acceptance Tests & Daemon Deployment

Objective: Verify complete system acceptance and enable background systemd service on Raspberry Pi.
Execution Command: pytest tests (runs all 54 tests across all suites).
Daemon Enable Commands (Pi):
sudo systemctl enable --now secureflex-receiver.service
sudo systemctl status secureflex-receiver.service
Pass Criteria: 100% of tests pass; receiver daemon starts automatically on boot and recovers safely.

10. Operator Runbook: Step-by-Step Live Execution

Follow this operational procedure during live testing between the Windows laptop and Raspberry Pi 5:

Step	Operation	System / Console	Exact Shell Command	Verification Check
1	**Key Gen**	Windows Laptop	python config/key_generator.py -o config/secret.key	Generates 16-byte hex key.
2	**Key Copy**	Windows Laptop	scp config/secret.key pi@10.41.237.148:~/SecureFlex-MAVLink/config/	Key transferred out-of-band.
3	**Secure Perms**	Raspberry Pi 5	chmod 600 ~/SecureFlex-MAVLink/config/secret.key	Readable only by service.
4	**Start Receiver**	Raspberry Pi 5	python raspberry_pi/secure_receiver.py --port 14550	Prints 'Listening on 0.0.0.0:14550'.
5	**Start Sender**	Windows Laptop	python windows/secure_transmitter.py --target-ip 10.41.237.148	Sends heartbeats at 1.0 Hz.
6	**Verify Link**	Raspberry Pi 5	*Observe console output*	FSM: IDLE -> RUN; displays HEARTBEAT.
7	**Tamper Test**	Windows Laptop	python windows/secure_transmitter.py --test-corrupt-ct	Pi rejects at Stage 6; tag failure.
8	**Replay Test**	Windows Laptop	python experiments/fault_injector.py --fault replay	Pi rejects at Stage 5; REPLAY_ATTACK.

11. Future FPGA Acceleration & Hardware Migration Pathway

With the software prototype validated on Windows and Raspberry Pi 5, the project prepares for hardware acceleration on an FPGA coprocessor (e.g., AMD Xilinx Zynq-7000 or UltraScale+):

1. Partitioning of Responsibilities: Raspberry Pi 5 retains responsibility for Wi-Fi/UDP networking, system management, persistent logging, and high-level flight telemetry. The FPGA accelerates the 320-bit Ascon permutation, round constants, and real-time S-box parity evaluation over an SPI or AXI-Lite bus.

2. True Combinational Output Clamp: The FPGA implementation introduces a single-cycle combinational kill-switch that zeros the plaintext bus instantly upon any parity mismatch, preventing transient fault leakage through side channels.

3. Physical Fault Injection Hardening: Hardware implementation enables physical electromagnetic fault injection (EMFI) and clock glitch testing to rigorously evaluate physical hardware resilience against hostile electronic warfare.

PROJECT ACCEPTANCE & COMPLETION CONFIRMATION

The SecureFlex-MAVLink software prototype meets all requirements specified in the Implementation Guide. All 54 test cases pass, official NIST SP 800-232 known-answer vectors verify with 100% precision, S-box parity models achieve 100% fault detection, and the live socket loopback verifies robust operation.